CHAINBREAKER

Safety-aware incident reconstruction and containment planning for connected-city infrastructure

HACKATHON ChronoForge **TRACK** Cybersecurity **TEAM** PVR **MEMBERS** Kumar Raunak; Vishesh Sharma **THEME** Cities of the Future

1. Problem and Motivation

Cities of the Future run on connected infrastructure: traffic signal controllers, smart meters, EV chargers, CCTV, public Wi-Fi and the operator portals that manage them. A single stolen operator credential can now reach physical services. The evidence of such an attack is scattered across staff authentication, DNS, IoT message logs and application logs, and the municipal, campus and industrial-park IT teams who must investigate it have no full SIEM and no OT analysts.

Two gaps remain unsolved. First, **correlation breaks in cities**: linking events by shared IP or account merges unrelated activity through public-Wi-Fi NAT, shared operator accounts and vendor gateways. Second, **the standard response can do more harm than the attack**: isolating a traffic controller or disabling a shared control account may disrupt life-safety services. Existing tools do not weigh that cost. IoT devices are also unusually predictable (fixed peers, periodic telemetry, a small command set), which generic tools do not exploit.

> **Problem statement.** How can a small city or campus team reconstruct a multi-stage intrusion across IT and IoT logs, and choose the containment action that stops it with the least harm to safety-critical services, while a human approves every real-world action?

2. Target Users and Use Cases

Users	Use cases
Municipal and smart-city IT operators; campus IT teams (a campus is a city in miniature); industrial-park and utility admins; students learning incident response	(1) Triage a compromised operator account that reaches an IoT gateway. (2) Spot a rogue or hijacked IoT device from its behaviour. (3) Choose containment under service-criticality limits. (4) Produce an audit-ready, verifiable incident report.

3. Fit with the Cybersecurity Track

Track area	How ChainBreaker addresses it
Security monitoring and incident analysis tools	Core product: multi-source correlation, attack-chain reconstruction, timeline and case report
IoT and connected-device security	Per-device-class behavioural fingerprints and command-plane anomaly checks
Network traffic analysis and anomaly detection	DNS entropy, rate and beacon-periodicity detectors; rare-destination tracking
Phishing, scam and malicious URL detection	Look-alike domain detection against the city's own domains; URL indicators in portal logs
Privacy-preserving applications; AI-assisted analysis	Keyed pseudonymization of MACs, IPs and accounts; local processing; optional LLM summary with verified row citations

4. Proposed Solution and Key Features

ChainBreaker turns uploaded CSV/JSONL logs into a **temporal city event graph** (accounts, hosts, IoT devices, domains and sessions as nodes; time-stamped events as edges) with five layers.

A. Probabilistic evidence linking (fixes the NAT / shared-account problem)

A link between two events through a shared entity *e* gets a Fellegi-Sunter-style weight from entity rarity and time decay: $w(e, dt) = -\log p(e) \times \exp(-dt / T_{type})$. A rare external IP is strong evidence; a public-Wi-Fi NAT address carries almost none. Weights map to link probabilities by isotonic calibration (Brier score reported), and the UI shows the breakdown for every link. Per-source clock skew and host/account aliases are corrected first, and each correction is listed and reversible.

B. IoT behavioural fingerprints

From a baseline, each device class (signal controller, meter, camera, charger) gets a profile: allowed peers, message period, payload-size distribution and command types. Deviations are scored with z-scores on period and size plus set-novelty on peers and commands. Control commands outside maintenance windows or from unusual sources raise findings.

C. Stage-aware attack-path inference

Findings are tagged with MITRE ATT&CK (Enterprise and ICS) stages. A Viterbi-style dynamic program over each incident DAG maximizes $\sum w(\text{link}) + \sum \log P(\text{stage}_{k+1} \mid \text{stage}_k)$, returning the top-k ordered chains. A greedy **minimal-justification** step finds the smallest set of rows that still supports the incident.

D. Safety-aware containment planner (read-only)

Each asset carries a criticality tier (life-safety, essential, convenience) from a device inventory. For every candidate action (revoke session, block a command or topic, isolate host, block domain) the planner computes a weighted minimum vertex cut and a **collateral cost** from the baseline legitimate services it would disrupt. It shows a Pareto frontier of evidence severed vs collateral, never auto-recommends isolating a life-safety asset (it proposes narrower options such as blocking one command), and runs a **counterfactual replay** that re-infers paths with the entity removed to show the residual attack and any alternative route. Nothing is executed; the analyst approves and the decision is logged.

E. Verifiable evidence and red-team testing

Evidence rows are hashed into a **Merkle tree** so a single row can be proven part of the exported report. A synthetic generator with evasion knobs (rate, jitter, source distribution, trusted-domain C2) and a hill-climbing attacker finds settings that evade the detectors, which we use to harden the rules.

5. Detection Layer (statistical, explainable, rule-ID tagged)

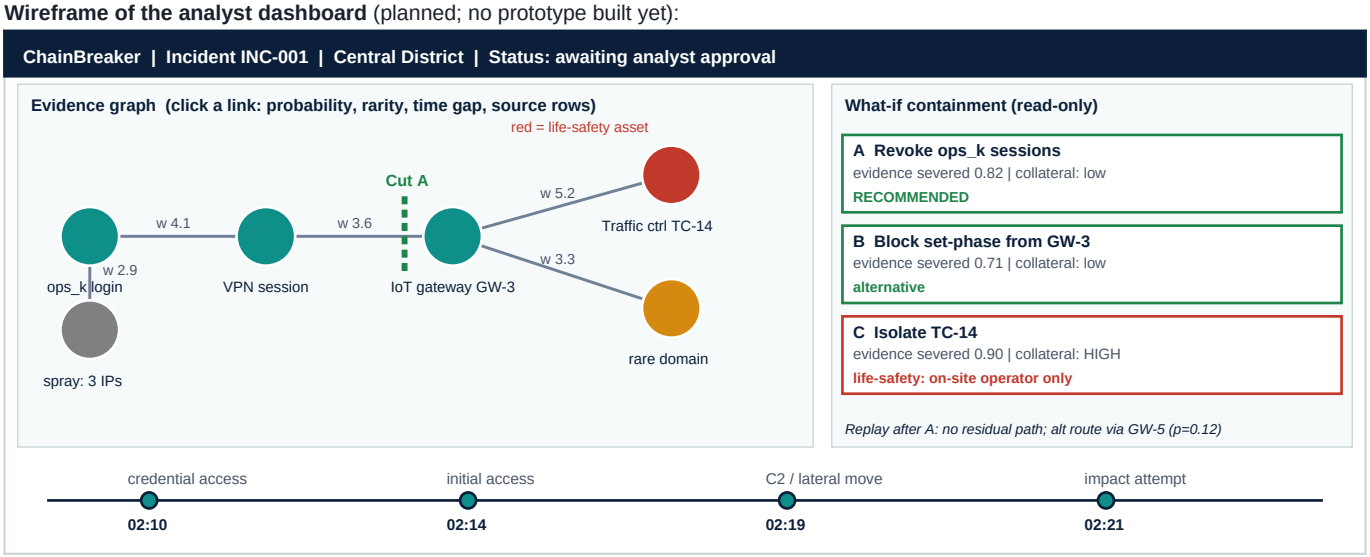
Source	Detectors
Staff authentication	Beta-binomial failure-rate test per account and source; distinct-accounts-per-source for spraying; CUSUM for low-and-slow; failures-then-success sequence
DNS / network	Character n-gram and entropy score for DGA-like names; EWMA z-score on query rate; inter-arrival coefficient of variation for beaconing; first-seen and count-min-sketch rarity
IoT message logs	Device-class fingerprint deviation; unauthorized command or topic; off-window control commands; new peer
Portal / web	Look-alike domain detection (edit distance to the city's domains); URL indicators (encoded payloads, traversal); per-host bursts

Each finding carries a rule ID, the features that fired and links to source rows. Heuristic scores are never shown as the probability of compromise; only the calibrated link probabilities in 4A are, and only on our test data.

6. Architecture and Technology Stack

Layer	What it does	Implementation
1. Ingest	Validate, normalize timestamps and schema, pseudonymize, Merkle-hash rows, flag bad rows	Python, pandas, pydantic
2. Resolve and detect	Skew and alias correction; rules, statistical detectors and IoT fingerprints	Python, NumPy, SQLite
3. Infer and plan	Probabilistic linking, Viterbi paths, minimal justification, min-cut planner, replay	networkx, scikit-learn
4. Test	Smart-city scenario generator, evasion search, metrics and ablations	Python
5. Explain and report	Evidence graph, timeline, link inspector, what-if panel, verifiable report	FastAPI, React + Vite, Cytoscape.js

Wireframe of the analyst dashboard (planned; no prototype built yet):



7. Innovation and Differentiation

Typical log dashboard or SIEM	ChainBreaker
Lists alerts per rule	Infers the most plausible ordered attack chain across IT and IoT sources, with a minimal justification
Links on shared fields (NAT and shared accounts cause false merges)	Rarity- and time-weighted link probabilities, inspectable link by link
Generic response checklist; ignores what a shutdown breaks	Safety-tiered, Pareto-ranked containment with counterfactual replay; life-safety assets protected
Treats IoT like any host	Device-class behavioural fingerprints and command-plane checks
Accuracy asserted	Tested against benign near-misses and an evading attacker, with calibration and ablations

Judging criteria map: originality (4A, 4D), track relevance (section 3), technical depth (4A-4E), feasibility (sections 8-9), impact (section 10).

8. Evaluation Plan

A **synthetic smart-city testbed** generates labelled logs from staff SSO/VPN, DNS, MQTT-style IoT messages and a city portal. **Attack scenarios:** (1) password spray, valid login, pivot to an IoT gateway, an off-window command to a traffic controller, DNS beaconing; (2) a hijacked camera beaconing to a rare domain, then reaching an EV-charger manager. **Benign near-misses:** scheduled firmware push, rush-hour telemetry spike, event-day public-Wi-Fi NAT surge, contractor bulk login. **Metrics:** event precision/recall/F1, incident clustering (adjusted Rand index), correct-chain rate, calibration (Brier), time to detect, events per second. **Ablations:** naive vs weighted linking, stage prior on/off, skew correction on/off, with 10-30% dropped fields. Results apply to synthetic data only; if time allows we repeat on a public IoT dataset such as TON_IoT or CIC IoT 2023.

9. Feasibility and 24-Hour Implementation Plan

Hours	Work	Output
0-3	Event schema, device inventory format, scenario and ground-truth spec	Schema, spec
3-7	Ingestion, hashing, pseudonymization; scenario generator	Labelled datasets
7-11	Detectors and IoT fingerprints with rule IDs and source links	Findings
11-16	Alias handling, weighted linking, incident clustering, Viterbi paths	Incidents, chains
16-19	Safety-aware planner and replay; dashboard views	Ranked actions, UI
19-23	Report export, evaluation harness, ablations	Metrics, report
23-24	Rehearsal, demo data, architecture and limits slides	Demo-ready build

Core path: ingest, detect, weighted linking, path inference, planner and dashboard. **Cut first if behind:** evasion search, skew estimation, calibration, Merkle proofs (fall back to a hash chain), then the LLM summary. Standard open-source libraries keep each stage small, and one member can own detection and inference while the other owns planner, UI and evaluation.

10. Expected Impact and Scalability

Impact: small cities, campuses and industrial parks get faster, explainable triage and containment advice that respects public safety, plus a practical teaching tool for incident response. **Scalability:** ingestion is partitioned by district or site and the graph is updated incrementally over sliding windows; adapters can read syslog, Zeek-style DNS and broker logs; findings can be exported as Sigma-style rules or STIX, and the planner can feed existing SIEMs such as Wazuh. These are directions after the hackathon, not promises for the build.

11. Responsible Use and Risks

ChainBreaker is a defensive, read-only tool for synthetic, public or explicitly authorized logs. It will not scan third-party systems, exploit vulnerabilities, or block traffic or disable accounts automatically; every containment option needs human approval, and life-safety assets need an on-site operator. MACs, IPs and accounts are pseudonymized with a keyed HMAC, data stays local, and retention is low. The red-team generator creates synthetic log events only, not attack tooling. Main risks are scope (fixed priority order above) and tuning to our own synthetic data (fit on one set, test on held-out sets, state limits). A finding is a reason to investigate, not proof of compromise.

Proposed Demo for Judges

1. Upload a labelled city dataset (normal activity plus the spray-to-traffic-controller scenario). **2.** Show the ordered chain, its minimal justification and any link's probability breakdown. **3.** Toggle naive linking: public-Wi-Fi NAT falsely merges a benign surge; the weighted model removes it. **4.** Open the what-if panel: isolating the traffic controller is flagged life-safety, revoking the operator session is recommended, and the replay shows the residual path. **5.** Show metrics and ablations, then export the case report and verify one row against the Merkle root.